Deep Residual Networks (ResNet)

Architecture, Internals & Integration Patterns

Deep Learning Architecture Series

"Deep Residual Learning for Image Recognition"
He et al., CVPR 2016

2024

Outline

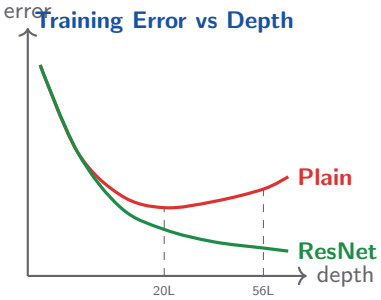
1. Motivation
2. Residual Block
3. Full Architecture
4. Inside the Block
5. Variants
6. Plugging ResNet
7. Practical Guidance

The Degradation Problem

Key Observation (He et al., 2016)

Adding more layers to a plain deep network **degrades** training accuracy — not due to overfitting, but due to **optimisation difficulty**.

- A 56-layer plain net performs *worse* than a 20-layer net on CIFAR-10
- Vanishing/exploding gradients still present despite BatchNorm
- The hypothesis: very deep networks are hard to optimise to an identity mapping

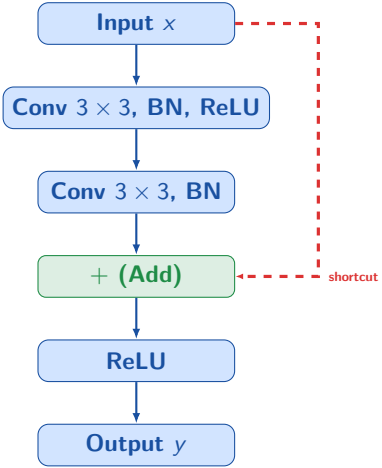


Residual Learning Idea

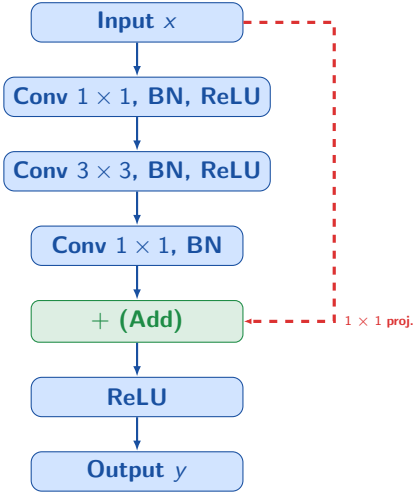
Instead of learning $\mathcal{H}(x)$ directly, learn the **residual**:

The Residual Building Block

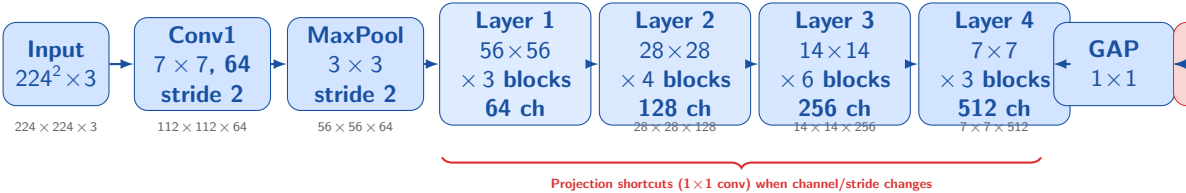
Basic Block (ResNet-18/34)



Bottleneck Block (ResNet-50/101/152)



ResNet Full Architecture (ResNet-50 walkthrough)



Stem (Conv1 + Pool)

Large receptive field fast downsampling.
$224 \rightarrow 56$ in two steps.

4 Residual Stages

Each stage doubles channels, halves spatial resolution (stride-2 first block).

Head (GAP + FC)

Global Average Pooling removes spatial dims; FC maps to class logits.

Inside a Residual Block — Step by Step

Forward pass through one bottleneck block:

1. 1×1 **Conv (reduce)**: Compress channels: $C \to C/4$. Cheap operation acting as a learned feature selector.
2. 3×3 **Conv (mix)**: Spatial convolution in the compressed space. Captures spatial patterns efficiently.
3. 1×1 **Conv (expand)**: Restore channels: $C/4 \to C$. Acts as a learned projection back.
4. **BatchNorm after each conv**: Stabilises activations, enables high LR, acts as regulariser.
5. **Shortcut Add**: Element-wise addition. Provides gradient highway: $\nabla \mathcal{L}$ flows directly through the skip connection.
6. **ReLU after Add**: Applied once, not inside the

Gradient Flow

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \underbrace{\left(1 + \frac{\partial \mathcal{F}}{\partial x}\right)}_{\text{never vanishes to 0}}$$

Projection Shortcut

When stride>1 or channels change, use a 1×1 conv on the skip path to match dimensions:

$$y = \mathcal{F}(x, W_i) + W_s x$$

Pre-Activation Variant (He et al. 2016b)

BN, ReLU *before* each conv

ResNet Family — Variants & Key Numbers

Model	Blocks	Block type	Params	GFLOPs	Top-1 (%)	Top-5 (
ResNet-18	[2,2,2,2]	Basic	11.7M	1.8	69.8	89.1
ResNet-34	[3,4,6,3]	Basic	21.8M	3.7	73.3	91.4
ResNet-50	[3,4,6,3]	Bottleneck	25.6M	4.1	76.1	92.9
ResNet-101	[3,4,23,3]	Bottleneck	44.5M	7.8	77.4	93.5
ResNet-152	[3,8,36,3]	Bottleneck	60.2M	11.6	78.3	94.1
ResNeXt-50 (32×4d)	[3,4,6,3]	Grouped Bottleneck	25.0M	4.3	77.6	93.7
Wide ResNet-50 (2×)	[3,4,6,3]	Wide Basic	68.9M	11.4	78.5	94.1

■ ResNeXt

Introduces **grouped convolutions** (cardinality). More groups = wider network without quadratic cost. Replaces single 3×3 conv with G parallel groups.

↔ Wide ResNet

Increases **channel width** (multiplier k). Wider shallow nets outperform deeper narrow ones. Trades depth for width.

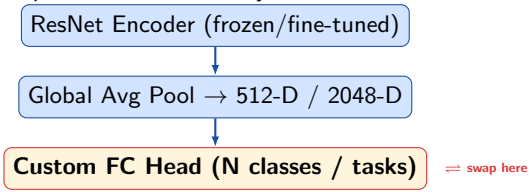
○ SE-ResNet

Adds **Squeeze-Excitation** channel attention after each block. Lightweight recalibration: global pooling → FC → sigmoid → scale.

ResNet as a Backbone — Plug-in Patterns

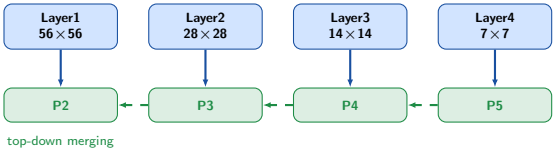
Pattern 1: Classification Head Swap

Replace the final FC layer with a custom head:

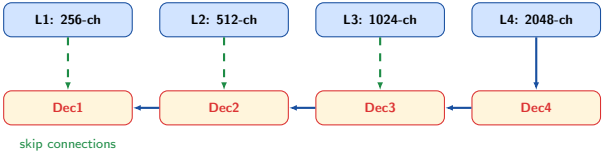


Pattern 2: Feature Pyramid (FPN)

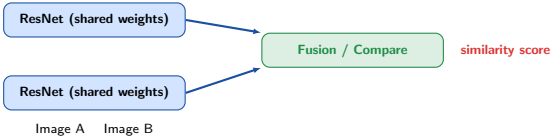
Tap into intermediate layers for multi-scale features:



Pattern 3: Encoder in U-Net (Segmentation)



Pattern 4: Two-stream / Siamese



Downstream Task Integration — Recipes

Task	What to attach	Which layers to tap	Training strategy
Image Classification	FC / MLP head	After GAP	Freeze backbone, train head; then fine-tune all
Object Detection	FPN + RPN + RoI Head (Faster R-CNN)	C2–C5 (Layers 1–4)	Pretrained backbone; custom LR for head
Semantic Segmentation	DeepLab ASPP or U-Net decoder	Layers 1–4 (multi-scale)	Dilated convs in Layer 3/4 (remove stride)
Instance Segmentation	Mask R-CNN mask head	FPN on C2–C5	Two-stage: detect then mask
Anomaly Detection	Autoencoder decoder	Layer 4 (or all)	Train decoder only; encoder frozen
Self-Supervised / SSL	Projector MLP (SimCLR, MoCo)	After GAP	Contrastive loss; discard projector later
Feature Extraction	None (use as-is)	Any intermediate	Freeze entirely

Practical Guidance — When & How to Use ResNet

✓ Choosing the Right Variant

- **ResNet-18/34**: fast prototyping, small datasets, edge deployment
- **ResNet-50**: best general-purpose backbone; accuracy/speed sweet spot
- **ResNet-101/152**: large-scale datasets where accuracy is critical
- **ResNeXt-50**: same params as ResNet-50, better accuracy via grouped convs
- **SE-ResNet**: add channel attention for small marginal cost

⋆ Key Hyperparameters

Weight init	Kaiming normal (He init)
Normalisation	BatchNorm (default)
Activation	ReLU / PreActResNet

>> PyTorch Quick-start

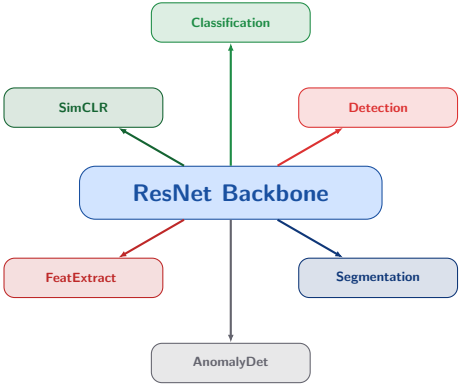
```
import torchvision.models as M

# Pretrained backbone
backbone = M.resnet50(weights=
  M.ResNet50_Weights.IMAGENET1K_V2)

# Swap head for N classes
in_f = backbone.fc.in_features
backbone.fc = nn.Linear(in_f, N)

# Tap intermediate features
feats = {}
backbone.layer3.register_forward_hook(
  lambda m,i,o:  feats.update({'l3':o}))

# Freeze encoder
for p in backbone.layer1.parameters():
  p.requires_grad = False
```

! Common Pitfalls

Summary

Core Contributions of ResNet

- **Residual / skip connections** solve the degradation problem
- Enables training of **100+ layer** networks reliably
- **Identity shortcut** = free gradient highway, zero extra parameters
- Bottleneck design: **3× cheaper** than naive deep stacking



Why It's Everywhere

- Strong ImageNet pretrained features transfer to nearly any visual task
- Modular: easy to swap heads, tap features, freeze stages
- Well-supported in every major framework

Key Formula

$$\boxed{y = \mathcal{F}(x, \{W_i\}) + x}$$

Simple. Powerful. Everywhere.